

SENTINEL

AI-Powered Edge-Cloud Video Intelligence & Dynamic Offloading Platform

PROJECT PROGRESS & REMAINING WORK REPORT

Status snapshot: 05 September 2026

This report summarizes the problem statement, architecture, technology stack, implementation completed so far, verification status, current blocker, and remaining roadmap for the Sentinel hackathon project.

Current verified engineering milestone

The Edge offloading subsystem reached a major milestone with a reported full regression of 154/154 tests passing. This included dynamic offloading, execution contracts, concrete executors, registry/router integration, telemetry, adaptive scheduling stability, and adaptive decision explainability.

1. Executive Summary

Sentinel is a distributed video-intelligence platform designed to process surveillance workloads close to the camera while intelligently distributing expensive AI workloads across Edge, Local and Cloud compute. The system combines real-time computer vision, temporal tracking, behavioral analysis, selective AI triggering, ALPR/threat intelligence, dynamic workload offloading and telemetry-driven scheduling.

The implementation has progressed substantially beyond the original foundation. The project now contains a working event pipeline, YOLO + ByteTrack tracking, rich track state, behavioral intelligence, selective-AI decision logic, ALPR/threat/context integration, dynamic offloading, execution abstractions, adaptive telemetry and explainable scheduler decisions.

Current position: Phase 4.3 Scheduler Decision Engine Hardening is in progress. A recent focused run collected 36 tests: 30 passed and 6 failed. The six failures share one integration cause: the scheduler invokes `validate_workload()` while telemetry mode wraps the base policy in `AdaptiveSchedulingPolicy`, which currently does not expose that delegated method.

Project objectives

- Reduce unnecessary video/data movement by processing close to the source.
- Maintain persistent object identities across frames.
- Detect suspicious behavior using temporal track intelligence.
- Run expensive AI selectively instead of continuously.
- Recognize license plates and connect observations to threat intelligence.
- Choose Edge, Local or Cloud execution based on workload and node conditions.
- Adapt scheduling using historical latency and execution reliability.
- Expose explainable, serializable decisions for future APIs and dashboards.
- Reach a reproducible, secure and measurable hackathon demonstration.

2. Problem Statement

Traditional video-surveillance pipelines can become expensive and difficult to scale when every frame is processed with heavy AI or when large amounts of video are continuously transmitted. Fixed compute placement also performs poorly when edge resources are busy, network latency changes, queues grow, or workloads have different urgency and resource requirements.

Sentinel addresses this by separating continuous lightweight processing from expensive intelligence. Detection and tracking remain near the camera; behavior, confidence, workload type and system state can determine whether deeper analysis is skipped, executed locally or routed to another compute tier.

Core decision problem

For a workload W and candidate nodes $N = \{\text{Edge, Local, Cloud}\}$, Sentinel must select a feasible execution node with the best overall suitability. The current policy explicitly scores resource headroom, estimated latency, queue utilization, workload priority, bandwidth and target preference.

End-to-end concept

Camera → Edge frame sampling → YOLO + ByteTrack → Track State → Behavior Analysis → Selective AI Decision Engine → Dynamic Offload Scheduler → Execution Router → Executor → Event Pipeline → FastAPI/Redis Worker → PostgreSQL → future Command Center.

Expected benefits

- Lower GPU/CPU use through selective AI.
- Lower network usage by avoiding unnecessary raw/full-resolution transmission.
- Low-latency treatment for urgent workloads.
- Elastic use of local/cloud capacity when edge resources are constrained.
- Better routing decisions as execution telemetry accumulates.
- Transparent scheduling decisions that can be inspected and serialized.

3. System Architecture

High-level architecture

```
Camera
↓
Edge Node – decoding, frame sampling, detection, tracking, behavior, selective AI, ALPR/threat processing
↓
Decision / Offload Layer – Edge / Local / Cloud
↓
Redis Streams / FastAPI
↓
Workers
↓
PostgreSQL
↓
WebSocket Command Center
```

Edge processing pipeline

- CameraStream acquires video frames.
- Frame sampling controls how often tracking inference runs.
- YOLO + ByteTrack generates detections and persistent track IDs.
- TrackStateManager enriches tracks with observations, movement and lifecycle state.
- BehaviorEngine analyzes temporal patterns.
- DecisionEngine decides whether expensive AI should run.
- OffloadScheduler selects an execution target.
- ExecutionRouter dispatches work to a registered executor.
- ALPR, threat intelligence and context fusion operate in the downstream processing path.
- Events are emitted through the event pipeline for persistence.

Verified event path

The established foundation proved the flow Camera → Edge → FastAPI → Redis Stream → Worker → PostgreSQL. The Redis consumer group and database persistence were previously verified as operational.

4. Technology Stack

Layer	Technology / Component	Role
Language	Python 3.11	Primary implementation language.
Vision	OpenCV	Video/frame acquisition and image processing.
Detection	YOLO	Real-time object detection.
Tracking	ByteTrack	Persistent multi-object track IDs.
Behavior	Custom Python Behavior Engine	Temporal behavioral rules.
Selective AI	Decision Engine	Skip/local/cloud analysis decision.
ALPR	ALPR + OCR pipeline	License-plate recognition.
Threat Intel	ThreatIntelEngine	Watchlist/risk-oriented checks.
Scheduling	SchedulingPolicy + AdaptiveSchedulingPolicy	Static and telemetry-adaptive node scoring.
Execution	EdgeExecutor / LocalExecutor / ExecutionRouter	Execution abstraction and dispatch.
Telemetry	ExecutionTelemetryCollector	Historical execution statistics.
API	FastAPI	Event ingestion/control-plane foundation.
Messaging	Redis Streams	Asynchronous event/work queues.
Database	PostgreSQL	Persistent event storage.
Testing	pytest	Unit, integration and regression testing.
Deployment	Docker Compose	Reproducible infrastructure foundation.
Frontend	React / Vite	Planned command-center interface.
Object storage	MinIO / S3	Planned evidence storage.

The architecture is modular: scheduling and execution are separated from the camera loop, while the event path can be tested independently. This separation is important for later distributed deployment.

5. Implementation Completed So Far

Phase	Module	Scope	Status
1.1	Foundation & Architecture	Structure, config, FastAPI, PostgreSQL, Redis, event schema.	COMPLETE
1.2	Event Pipeline	Edge → API → Redis → Worker → PostgreSQL.	COMPLETE
2.1	Object Tracking	YOLO + ByteTrack + persistent IDs.	COMPLETE
2.2-A	Tracking State	Speed, acceleration, trajectory, dwell/stationary state.	COMPLETE
2.2-B	Behavior Engine	Loitering, suspicious stop, rapid movement, direction change.	COMPLETE
2.2-C	Behavior Integration	Live Edge integration and event generation.	COMPLETE
2.3	Selective AI / Decision Engine	Selective expensive-AI decisions.	COMPLETE
3.x	ALPR / Threat / Context	ALPR path, threat checks and context fusion.	COMPLETE
4.1	Dynamic Offloading	Edge vs Local vs Cloud selection.	COMPLETE
4.2-A	Execution Contracts	Execution request/result abstractions.	COMPLETE
4.2-B	EdgeExecutor	Concrete Edge execution.	COMPLETE
4.2-C	LocalExecutor	Concrete Local execution.	COMPLETE
4.2-D	ExecutionRegistry	Executor registration and lookup.	COMPLETE
4.2-E	ExecutionRouter	Workload routing through executors.	COMPLETE
4.2-F	Edge Integration	Actual Edge main integration.	COMPLETE
4.2-G	Telemetry + Adaptive Scheduling	Telemetry, adaptation, stability and explainability.	COMPLETE
4.3	Scheduler Hardening	Validation, deterministic ranking, latency/deadline and safe fallback.	IN PROGRESS

Tracking and behavior

The tracking state evolved from simple identity/lifecycle fields to observations, first/last centers, total distance, speed, acceleration, trajectory, stopped-since time, missing frames, direction and lifetime. Four behavioral rules were implemented: loitering, suspicious stop, rapid movement and abnormal direction change.

Selective AI and offloading

The Edge loop feeds behavior and confidence information into the Decision Engine. The dynamic offloading layer then selects a target and routes execution through the execution abstraction. The live pipeline maintains offload counters for Edge, Local, Cloud, rejected and fallback decisions.

6. Dynamic Offloading & Adaptive Scheduling

Base scheduling policy

The base SchedulingPolicy evaluates capability and computes a deterministic suitability score. Current explicit weights are: resource 0.30, latency 0.25, queue 0.15, priority 0.15, bandwidth 0.10 and target preference 0.05.

Capability checks

- Node is available.
- Target is allowed by the workload.
- Queue is below maximum capacity.
- Required GPU availability is satisfied.
- Required memory headroom is satisfied.
- Minimum bandwidth requirement is satisfied.

Latency model

Estimated latency combines workload compute time, network latency, queue penalty and resource contention. This creates a common metric for comparing candidate nodes.

Adaptive scheduling

AdaptiveSchedulingPolicy decorates the base policy with historical execution telemetry. It uses node/workload-specific execution statistics and bounded latency/reliability adjustments so telemetry can influence routing without completely overriding the base policy.

Decision explainability

AdaptiveDecisionExplanation records the base score, latency adjustment, reliability adjustment, total adjustment, final score, telemetry sample count, success rate, average latency and whether adaptation was active. CandidateExplanation records rank, node, target, score and optional adaptive details. SchedulingResult serializes these structures for future API/dashboard use.

Verified milestone

The latest completed full Edge regression reported 154 passed in 8.07 seconds. Principal adaptive modules also compiled successfully at that checkpoint.

7. Current Work — Phase 4.3 Scheduler Decision Engine Hardening

Purpose

Phase 4.3 is intended to make scheduler decisions safer, more deterministic and more explicit before broader distributed deployment.

Hardening scope

- Workload validation for malformed or impossible profiles.
- Priority-aware enforcement for urgent workloads.
- Explicit deadline/latency feasibility handling.
- Centralized deterministic candidate ranking.
- Preferred-target selection with safe fallback.
- Failure-safe rejection instead of uncontrolled errors.
- Clearer decision reasons and rejection explanations.
- Edge-case coverage for empty registries, unavailable nodes, queue saturation and invalid constraints.

Latest test evidence

The latest focused command collected 36 tests: 30 passed and 6 failed. The failure summary shows the same `AttributeError` in all six failing paths: `AdaptiveSchedulingPolicy` has no `validate_workload` attribute.

Root cause

The scheduler was hardened to call the validation contract through its active policy. Without telemetry, the active policy is `SchedulingPolicy`. With telemetry, the scheduler wraps it in `AdaptiveSchedulingPolicy`. The wrapper delegates existing capability/scoring responsibilities but the new validation method has not yet been delegated.

Immediate fix

Add a thin `validate_workload()` delegation method to `AdaptiveSchedulingPolicy`, rerun the 36 focused tests, run the dedicated 4.3 tests, and then rerun the complete Edge regression. Do not weaken the new validation simply to make the tests pass.

8. Remaining Development Roadmap

Phase	Remaining work	Priority
4.3	Finish scheduler hardening and restore clean regression baseline.	NOW
5.1	Node/camera registry, health checks, configuration and model management.	HIGH
5.2	JWT authentication, users, roles and permissions.	HIGH
6.1	WebSocket command center, alerts and camera status.	HIGH
6.2	Map, camera locations and event heatmap.	MEDIUM
7.1	MinIO/S3 evidence snapshots and event evidence references.	HIGH
7.2	Metrics, latency, throughput and node-health observability.	HIGH
8.1	FPS, latency, batching and bandwidth-reduction benchmarks.	HIGH
8.2	Offline buffering, retries, idempotency and fault recovery.	HIGH
9.1	Complete Docker Compose demo deployment.	HIGH
9.2	Security, secrets, validation, logging and cleanup.	HIGH
10	Full end-to-end hackathon test matrix and demo scenario.	FINAL

Recommended execution order

- Fix the adaptive-policy validation delegation and complete Phase 4.3.
- Establish a clean full-regression baseline after 4.3.
- Complete the backend/control plane.
- Implement JWT + RBAC.
- Build/refine the WebSocket Command Center and geospatial views.
- Add evidence storage and operational observability.
- Benchmark latency, throughput, bandwidth reduction and failure behavior.
- Containerize the complete demo and execute the final end-to-end validation matrix.

9. Testing, Metrics & Definition of Done

Testing strategy

Testing has been used throughout the project. Coverage includes capability constraints, queue/GPU/bandwidth rejection, preferred-target fallback, critical workload routing, serialization, adaptive feedback, scheduler stability, explanation arithmetic, candidate explanations, execution contracts, executors, registry and router behavior.

Performance goals

Metric	Target	Status
Edge inference latency	<100 ms	Target; not yet proven for all workloads.
Bandwidth reduction	40-60%	Target; requires controlled baseline measurement.
Event loss	0	Requires failure/recovery validation.
Redis pending	0	Previously observed healthy; needs repeatable validation.
API errors	0 during healthy demo	Requires end-to-end validation.

Final hackathon definition of done

A scripted scenario should start from a camera/video source, detect and track an object, identify suspicious behavior, trigger selective AI, make and explain an Edge/Local/Cloud decision, execute the workload, emit and persist the event, display it in the operator interface, retain evidence, and survive the planned failure tests without uncontrolled crashes or event loss.

10. Final Project Assessment

Overall assessment: Sentinel has moved from a foundation prototype into a substantial modular edge-intelligence and dynamic execution framework. The strongest completed area is the Edge offloading subsystem, where contracts, executors, routing, telemetry, adaptive scheduling and explainability have been implemented and regression-tested.

Strong areas

- Established event-pipeline foundation.
- Persistent tracking and temporal track intelligence.
- Working behavioral analysis.
- Selective-AI decision logic.
- ALPR/threat/context integration in the Edge path.
- Dynamic Edge/Local/Cloud offloading.
- Execution contracts, executors, registry and router.
- Telemetry-driven adaptive scheduling.
- Explainable and serializable scheduler results.
- Large automated regression suite.

Remaining blockers to final demo readiness

- Finish Phase 4.3 and restore a clean regression baseline.
- Complete production-style control-plane management.
- Implement authentication and RBAC.
- Complete/refine the Command Center and geospatial interface.
- Add evidence storage and operational observability.
- Measure performance and reliability claims under controlled conditions.
- Deliver reproducible Docker deployment.
- Run the complete hackathon validation matrix.

Conclusion

The project is technically well positioned for the next stage. The immediate task is narrow and concrete: fix the adaptive-policy interface delegation introduced by Phase 4.3 hardening, validate the regression baseline, and then proceed to control-plane, security, dashboard, evidence, observability and final deployment work.

Report basis: project roadmap/progress notes, current scheduling/model/policy source, current Edge pipeline source, adaptive-policy source, and the latest supplied pytest output. Targets are explicitly labeled as targets where they have not yet been benchmarked.